UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 9 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Computer generations and history

Source: CSC429 (20).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Inventors of the devices:
(i) Mechanical Calculating Machine — Blaise Pascal (the Pascaline, 1642). [Note: Charles Babbage's Difference Engine is also a mechanical calculating machine, but the standard CSC429 answer for the 'Mechanical Calculating Machine' is Pascal.]
(ii) Analytical Machine (Analytical Engine) — Charles Babbage (1830s), often regarded as the father of the computer; Ada Lovelace wrote programs for it.
(iii) IAS Machine — John von Neumann (Institute for Advanced Study machine, the 'von Neumann machine', completed 1952), which established the stored-program concept.

(b) Later generations are designed as general-purpose machines because of the stored-program concept and advances in hardware. In early generations, machines were hard-wired for a specific task (e.g. ENIAC was rewired for each problem), so changing the function meant physically reconfiguring the machine — slow, expensive and inflexible. With the stored-program architecture (von Neumann), both instructions and data are held in the same memory, so the same hardware can run any program simply by loading different instructions. Combined with cheaper, smaller, more reliable and faster components (transistors, ICs, VLSI), this made it economic and practical to build one flexible machine that can solve many different problems (word processing, computation, control, AI, etc.) rather than many single-purpose machines. General-purpose designs therefore give greater flexibility, reusability, lower cost per task, and wider applicability.

Unofficial marking guide:

- *1 mark: (a)(i) Blaise Pascal (Pascaline)*
- *1 mark: (a)(ii) Charles Babbage (Analytical Engine)*
- *1 mark: (a)(iii) John von Neumann (IAS machine)*
- *1 mark: (b) stored-program concept — instructions and data in memory allow the same hardware to run different programs / early machines were hard-wired for a specific task*
- *1 mark: (b) flexibility/reusability/cost-effectiveness — one machine solves many problems, aided by cheaper, smaller, faster, more reliable components*

Model confidence: high.

Model note: OCR errors corrected conservatively: 'iiD' read as (iii); 'eadier' read as 'earlier'. For (a)(i), if the course specifically associates 'Mechanical Calculating Machine' with Babbage's Difference Engine, accept Charles Babbage; the conventional answer is Pascal's Pascaline. This is an unofficial model answer, not an official marking scheme.

QUESTION A2 (5 marks)

Topic: Data representation and number systems

Answer: 12.510 = 0x41480000 (sign=0, exponent=10000010, fraction=10010000000000000000000)

- *1 mark for the sign bit.*
- *1 mark for the biased exponent.*
- *1 mark for the 23-bit fraction.*
- *1 mark for the correct 32-bit grouping.*
- *1 mark for the final hexadecimal value.*

QUESTION A3 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The technique is direct addressing (also called absolute addressing). The instruction's address field A contains the actual/effective address of the operand, so the operand is fetched directly from memory location A — there is no intermediate address calculation or pointer dereference.

(b) How it is used: The instruction is fetched and its opcode decoded; the CPU takes the address field A as the effective address (EA = A) and places it on the address bus. The memory is read at location A,

© Hak Cipta Universiti Teknologi MARA

and the operand retrieved is passed to the ALU/register for execution of the instruction. For example, with LOAD A, the contents of memory word A are loaded into the accumulator. The address A is fixed at assembly/compile time, so the operand's location must be known when the program is written.

(c) Advantage: No address calculation is required — the effective address is obtained immediately from the instruction, so operand fetch is fast and simple (only one memory access for the operand). Disadvantage: The address field must be large enough to hold a full memory address, making the instruction long; also the address is fixed, so the technique offers limited flexibility (no relocation/indirection, restricted addressable range).

Unofficial marking guide:

• 1 mark: Correctly names the technique as direct addressing (absolute addressing).

• 1 mark: States that the address field A is the effective address of the operand (EA = A).

• 1 mark: Explains the fetch/execute step — address A placed on address bus, memory read at A, operand used by the instruction (e.g. LOAD A).

• 1 mark: Gives one valid advantage (e.g. no address calculation, fast/simple operand fetch, only one memory reference).

• 1 mark: Gives one valid disadvantage (e.g. long instruction/limited address range, fixed address so no flexibility/relocation).

Model confidence: high.

Model note: Figure 2 is inferred from the extracted text (Instruction: Opcode | Address A, then Memory: Operand), which matches direct addressing. If the figure actually shows an arrow from Address A to a memory word containing a further address, the answer would instead be indirect addressing; the text as given supports direct addressing.

QUESTION A4 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (7).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two advantages of SMP (Symmetric Multiprocessor) organization: (1) Performance/scalability — multiple processors share the same main memory and I/O, so several processes or threads can execute in parallel, increasing throughput and allowing the system to scale by adding processors. (2) Cost-effectiveness and simplicity — all processors are identical (symmetric) and share one copy of the OS, memory and I/O resources, so there is no need for separate memories or duplicated hardware, and a single operating system manages all processors uniformly. (Other acceptable advantages: fault tolerance/graceful degradation — if one processor fails the system can continue; and ease of load balancing since any processor can run any task.)

(b) Cluster organization: A cluster is a set of independent, complete computers (nodes, each with its own processor and memory) interconnected by a high-speed network, which cooperate and appear to users as a single, unified computing system. Each node runs its own copy of the operating system, and the cluster software coordinates the nodes to provide high availability and/or parallel processing.

(c) One benefit of cluster organization: High availability / fault tolerance — if one node fails, the remaining nodes take over its work, so the system continues to provide service (alternatively: high performance/scalability at low cost by adding commodity nodes, or better price/performance than a single large machine).

Unofficial marking guide:

• 1 mark: first valid advantage of SMP (e.g. increased throughput/parallel processing by sharing memory and I/O among multiple processors)

• 1 mark: second valid advantage of SMP (e.g. cost-effectiveness/simplicity of a single shared OS and memory, or graceful degradation)

• 1 mark: states that a cluster is a collection/group of independent computers (nodes)

• 1 mark: states that the nodes are interconnected by a network and cooperate/appear as a single system (or run separate OSes coordinated by cluster software)

• 1 mark: one valid benefit of clustering (e.g. high availability/fault tolerance, or high performance/scalability at low cost)

Model confidence: high.

Model note: Question text is clear; no OCR correction needed. Accept equivalent wording and other valid advantages/benefits (e.g. graceful degradation for SMP; low-cost scalability for clusters).

© Hak Cipta Universiti Teknologi MARA

QUESTION A5 **(10 marks)**

Topic: Input/output and interrupts

Source: CSC429 (2).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two types of keyboard characters [4 marks]

When a key is pressed, the keyboard does not send a character code directly; it sends a scan code that identifies the key's row and column position. The keyboard controller/driver then translates this scan code into a character code. Two useful types of keyboard characters are:

1. Printable (alphanumeric/graphic) characters: These are the ordinary visible characters such as letters (A–Z, a–z), digits (0–9), punctuation and symbols (e.g. ; , . / @ #). They are represented by a character code (commonly ASCII) and are stored in the keyboard buffer to be echoed to the display and passed to the running program. Example: pressing the 'A' key produces the ASCII code for 'A' (65 decimal / 0x41).

2. Control (non-printable) characters: These do not produce a visible glyph but cause an action or control function. They include control characters such as Enter/Carriage Return (CR, 0x0D), Line Feed (LF, 0x0A), Tab (0x09), Backspace (0x08), Escape (0x1B) and Delete. They are used to control the cursor, terminate a line of input, or signal an action to the program. Example: pressing Enter generates CR/LF, which terminates the input line and moves the cursor to a new line.

(An acceptable alternative pairing is: character keys that generate ASCII character codes vs. function/modifier keys such as Shift, Ctrl, Alt, F1–F12, which generate special scan codes or modify the interpretation of other keys.)

(b) Three categories of input/output module [6 marks]

An I/O module is the interface between the CPU/memory and a peripheral. Its main functions are to control the device, buffer data, perform error detection and present a standard interface (status, control and data registers) to the processor. Three categories, with examples, are:

1. Human-readable / user-interface modules: These connect the computer to a human operator, converting between human-perceptible form and digital signals. They handle relatively low data rates and often involve character or pixel data. Example: a keyboard controller (input) or a display/monitor adapter (output).

2. Machine-readable modules: These connect to equipment that reads or writes machine-oriented media, typically at higher speed and in block form. Example: a disk controller (magnetic/optical disk drive) or a magnetic tape/SSD controller.

3. Communication modules: These connect the computer to a remote device or network, converting between the computer's parallel internal form and the serial form used on the communication link. Example: a network interface card (Ethernet), a modem, or a serial (UART) port.

(Alternative acceptable classification: input modules, output modules and input/output (bidirectional) modules, e.g. keyboard, printer/display, and disk/network controller respectively.)

Unofficial marking guide:

- *1 mark: identifies first type of keyboard character (e.g. printable/alphanumeric or character keys)*
- *1 mark: explains it correctly, including that a scan code is translated to a character code such as ASCII and gives a valid example*
- *1 mark: identifies second type of keyboard character (e.g. control/non-printable or function/modifier keys)*
- *1 mark: explains it correctly with a valid example (e.g. Enter producing CR/LF, or Shift/Ctrl modifying another key)*
- *1 mark: names first I/O module category (e.g. human-readable)*
- *1 mark: describes it and gives an appropriate example (e.g. keyboard controller or display adapter)*
- *1 mark: names second I/O module category (e.g. machine-readable)*
- *1 mark: describes it and gives an appropriate example (e.g. disk controller)*
- *1 mark: names third I/O module category (e.g. communication)*
- *1 mark: describes it and gives an appropriate example (e.g. NIC/modem/UART)*

Model confidence: medium.

Model note: The question wording 'types of keyboard characters' is ambiguous; the model answer gives the printable vs control-character interpretation and notes the acceptable alternative (character keys vs function/modifier keys). The I/O module categories are not uniquely defined in the syllabus; the human-readable/machine-readable/communication classification is used, with the input/output/bidirectional classification noted as an acceptable alternative. No figure or table is required.

© Hak Cipta Universiti Teknologi MARA

QUESTION A6 (10 marks)

Topic: Data representation and number systems

a) Answer: 291010 = B5E16

- *1 mark for repeated division by 16.*
- *1 mark for correct remainders.*
- *1 mark for reading remainders in reverse order.*
- *1 mark for the correct hexadecimal digits.*
- *1 mark for the base subscript.*

b) Answer: 29016 = 65610

- *1 mark for place values 16^2, 16^1 and 16^0.*
- *1 mark for correct digit values.*
- *1 mark for correct products.*
- *1 mark for correct addition.*
- *1 mark for the final decimal value.*

QUESTION A7 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (10).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Four access methods for storage devices, each with an example:

1. Sequential access — data is read in a fixed linear order from start to end; to reach a record you must pass through all preceding records. Example: magnetic tape.

2. Direct access — data is organised into blocks/records that can be jumped to directly, but within a block access is sequential. Example: magnetic disk (hard disk drive), where the head seeks to a track/block and then reads sequentially.

3. Random access — any location can be accessed in (approximately) the same time, independent of its position. Example: main memory (RAM / semiconductor memory).

4. Associative access — data is accessed by matching its content (a key) rather than by address; a search is performed in parallel over the stored data. Example: cache memory (associative/TLB-style content-addressable memory).

(b) With 11 address bits, the number of addressable memory locations = 2^n = 2^11 = 2048 locations (i.e. 2K locations).

(c) 2 MB = 2 × 2^20 bytes = 2^21 bytes. Number of address bits = log2(2^21) = 21 bits (assuming byte-addressable memory, 1 byte per location).

Unofficial marking guide:

- *1 mark: correctly naming sequential access*
- *0.5 marks: correct example for sequential access (magnetic tape)*
- *1 mark: correctly naming direct access*
- *0.5 marks: correct example for direct access (magnetic disk)*
- *1 mark: correctly naming random access*
- *0.5 marks: correct example for random access (RAM/main memory)*
- *1 mark: correctly naming associative access*
- *0.5 marks: correct example for associative access (cache/CAM)*
- *2 marks: 2^11 = 2048 memory locations (1 mark for using 2^n, 1 mark for the value 2048)*
- *2 marks: 2MB = 2^21 bytes and 21 address bits (1 mark for correct conversion to 2^21, 1 mark for log2 giving 21 bits)*

Model confidence: high.

Model note: Assumes byte-addressable memory for part (c); if word-addressable with a stated word size, the bit count would differ. Part (a) accepts equivalent valid examples (e.g. CD-ROM for direct/random access, optical disk, or content-addressable memory for associative).

QUESTION A8 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (17).pdf, page(s) 3

© Hak Cipta Universiti Teknologi MARA

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two types of bus timing [4 marks]

Synchronous bus timing: All bus transactions are governed by a fixed, common clock shared by the master and slave. Events occur at defined clock edges, so the timing of address, data and control signals is predetermined. It is simple and fast for devices of similar speed, but the bus must run at the speed of the slowest device, and it cannot easily accommodate devices with widely differing response times.

Asynchronous bus timing: There is no common clock; instead, a handshake protocol (e.g. Master Ready / Slave Acknowledge, or request-acknowledge lines) coordinates the transfer. The master places the address/data and asserts a request, and the slave signals acknowledgement when it has completed the operation. This allows fast and slow devices to share the bus and adapts to device speed, but it requires more control lines and involves extra handshake overhead, making it slower per transaction than a well-matched synchronous bus.

(b) Three CPU components [6 marks]

1. Control Unit (CU): Directs the operation of the CPU. It fetches and decodes instructions and generates the timing and control signals that tell the ALU, registers and memory/IO which operations to perform, sequencing the fetch-decode-execute cycle.

2. Arithmetic Logic Unit (ALU): Performs arithmetic operations (add, subtract, etc.) and logical operations (AND, OR, NOT, XOR, shifts, comparisons) on data held in registers, and sets condition/status flags (zero, carry, sign, overflow).

3. Registers: Small, fast storage inside the CPU. They include the Program Counter (PC), Instruction Register (IR), Memory Address Register (MAR), Memory Data Register (MDR) and general-purpose/accumulator registers, used to hold the current instruction, addresses, operands and intermediate results during execution.

Unofficial marking guide:

- *1 mark: correctly names synchronous bus timing and explains it is driven by a common clock*
- *1 mark: states a valid advantage/limitation of synchronous timing (e.g. limited by slowest device)*
- *1 mark: correctly names asynchronous bus timing and explains the handshake/request-acknowledge mechanism*
- *1 mark: states a valid advantage/limitation of asynchronous timing (e.g. accommodates mixed-speed devices, extra overhead)*
- *1 mark: names and correctly describes the Control Unit and its role in fetching/decoding and generating control signals*
- *1 mark: names and correctly describes the ALU and its arithmetic/logical role*
- *1 mark: names and correctly describes registers (PC/IR/MAR/MDR/general-purpose) and their role*
- *2 marks: accuracy, completeness and use of correct terminology across the three component descriptions (awarded holistically)*

Model confidence: high.

Model note: Question text is clear; no OCR correction needed. Marking assumes 2 marks per bus-timing type (description + advantage/limitation) and 2 marks per CPU component (name + description), with the final 2 marks for overall accuracy/terminology.

QUESTION A9 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (21).pdf, page(s) 3, 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Highest-priority interrupt: Interrupt handler W. Justification: In a nested-interrupt timing diagram the handler that is entered first and is never pre-empted by any other handler has the highest priority. W begins first and runs to completion while X, Y and Z are all nested inside it, so W has the highest priority.

(b) Definition of interrupt: An interrupt is a signal (hardware or software) sent to the CPU that temporarily suspends the currently executing program, transfers control to a dedicated interrupt handler/service routine, and after the handler finishes returns control to the interrupted program at the point where it was suspended. It is used to handle asynchronous events such as I/O completion, timers and errors without polling.

(c) Lowest-priority interrupt: Interrupt handler Z. Justification: Z is the last handler to be entered and the first to be completed; it is nested inside W, X and Y and is never allowed to pre-empt any of them, so it has the lowest priority.

(d) Completion sequence under the defined (nested/priority) concept: Z, then Y, then X, then W. Because

© Hak Cipta Universiti Teknologi MARA

a higher-priority handler can pre-empt a lower-priority one, the innermost/lowest-priority handler Z finishes first, followed by Y, X and finally W.

(e) Completion sequence with nested interrupts disabled: The first interrupt to be serviced (W) runs to completion with all other interrupts masked/held pending; then X, then Y, then Z are serviced in turn. Completion sequence: W, X, Y, Z.

Unofficial marking guide:

- *1 mark: (a) identifies W as highest priority*
- *1 mark: (a) justifies by W being entered first and not pre-empted / all others nested within it*
- *1 mark: (b) defines interrupt as a signal that suspends the current program and transfers control to a handler*
- *1 mark: (b) states return to the interrupted program after the handler completes (or equivalent)*
- *1 mark: (c) identifies Z as lowest priority*
- *1 mark: (c) justifies by Z being entered last and completed first / never pre-empting others*
- *1 mark: (d) gives correct completion order Z, Y, X, W*
- *1 mark: (d) explains that lower-priority/innermost handlers complete first under nesting*
- *1 mark: (e) gives correct completion order W, X, Y, Z*
- *1 mark: (e) explains that with nested interrupts disabled each handler runs to completion before the next is serviced*

Model confidence: medium.

Model note: Figure 2 is only available as extracted text; the answer assumes the standard nested-interrupt timing diagram in which W is entered first and is not pre-empted, and X, Y, Z are nested inside it in that order. If the actual figure shows a different entry order, the priority ranking and completion sequences should be read directly from the figure. The OCR text places the definition of interrupt after the figure caption; it is treated as part (b) because the subpart list labels (b) with 2 marks and no text.

QUESTION A10 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (15).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Phases in the machine cycle [4 marks]

The machine cycle (also called the instruction cycle) consists of the following phases:

1. Fetch — the next instruction is fetched from main memory into the CPU, using the Program Counter (PC) to supply the address and the Memory Address Register (MAR) / Memory Data Register (MDR) to hold the address and the retrieved instruction. The PC is then incremented.
2. Decode — the Control Unit decodes the instruction held in the Instruction Register (IR) to determine the operation and its operands.
3. Execute — the required operation is carried out by the ALU and/or other CPU components (arithmetic/logic operation, data movement, branch, etc.).
4. Store / Write-back (and/or Interrupt check) — the result is written back to a register or to main memory, and the cycle checks for pending interrupts before returning to Fetch for the next instruction.

(b) Function of the different CPU components in the machine cycle [4 marks]

- Program Counter (PC): holds the address of the next instruction to be fetched; incremented during/after fetch so the next instruction is located.
- Memory Address Register (MAR): holds the address of the memory location to be accessed (instruction fetch or data read/write); connected to the address bus.
- Memory Data Register (MDR): buffers data/instructions passing between the CPU and main memory (holds the fetched instruction or the data read/written).
- Instruction Register (IR): holds the instruction currently being decoded/executed.
- Control Unit (CU): decodes the instruction in the IR and generates the control signals that sequence and coordinate all other components through the fetch–decode–execute–store phases.
- Arithmetic Logic Unit (ALU): performs arithmetic and logical operations during the execute phase; results go to the accumulator/registers.
- Registers / Accumulator: hold operands and intermediate/final results during execution and write-back.
- Internal buses: carry addresses, data and control signals between these components so the phases proceed in order.

(c) If access to main memory is blocked [2 marks]

The fetch phase cannot complete: the CPU cannot read the next instruction from memory into the IR, so the instruction cannot be decoded or executed. The machine cycle stalls/halts (the CPU waits, possibly raising a memory/fault exception or entering a halt state), and no useful instruction can progress until memory access is restored. (If the current instruction and its operands were already in registers/cache,

that one instruction might finish, but the next fetch would still stall.)

Unofficial marking guide:

• 1 mark: correctly naming Fetch as a phase
• 1 mark: correctly naming Decode as a phase
• 1 mark: correctly naming Execute as a phase
• 1 mark: correctly naming Store/Write-back (or interrupt check) as a phase
• 1 mark: correct function of PC and/or MAR/MDR in fetch
• 1 mark: correct function of IR and Control Unit in decode/control
• 1 mark: correct function of ALU/registers in execute
• 1 mark: correct function of write-back/registers or internal buses coordinating the cycle
• 1 mark: states fetch cannot complete / CPU stalls or halts because instruction cannot be read
• 1 mark: explains consequence (no decode/execute, cycle cannot progress until memory access restored)

Model confidence: high.

Model note: Question text is clear; no OCR correction needed. Some textbooks split the cycle into fetch/decode/execute only, or add an interrupt phase; the answer lists fetch, decode, execute and store/write-back (with interrupt check) to cover the 4-mark listing. Part (c) allows the caveat that an instruction already resident in registers/cache could finish, but the next fetch still stalls.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (18).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption: Figure 2 is interpreted as y = A - ((2C * D) / (B + A^2)). The expression is evaluated as: temp1 = 2*C; temp2 = temp1*D; temp3 = A*A; temp4 = B + temp3; temp5 = temp2 / temp4; y = A - temp5. Registers A, B, C, D are fixed; x, y, z are temporary. Final result must be in y.

One-address program (accumulator-based):

```
LOAD A       ; ACC = A
MUL A        ; ACC = A^2
ADD B        ; ACC = A^2 + B
STOR z       ; z = A^2 + B
LOAD C       ; ACC = C
MUL D        ; ACC = C*D
ADD C        ; ACC = 2*C*D  (since C*D + C*D = 2*C*D)
DIV z        ; ACC = (2*C*D)/(A^2+B)
STOR x       ; x = (2*C*D)/(A^2+B)
LOAD A       ; ACC = A
SUB x        ; ACC = A - x
STOR y       ; y = final answer
```

Two-address program (operand1 = operand1 op operand2):

```
MOV x, C     ; x = C
ADD x, C     ; x = 2C
MUL x, D     ; x = 2C*D
MOV z, A     ; z = A
MUL z, A     ; z = A^2
ADD z, B     ; z = B + A^2
DIV x, z     ; x = (2C*D)/(B + A^2)
MOV y, A     ; y = A
SUB y, x     ; y = A - x
```

Three-address program (dest = src1 op src2):

```
MUL x, C, D   ; x = C*D
ADD x, x, x   ; x = 2*C*D
MUL z, A, A   ; z = A^2
ADD z, B, z   ; z = B + A^2
DIV x, x, z   ; x = (2*C*D)/(B + A^2)
SUB y, A, x   ; y = A - x
```

Unofficial marking guide:

- *1 mark: correct interpretation of Figure 2 as y = A - ((2C*D)/(B + A^2)) and correct order of operations.*
- *6 marks: one-address program – 1 mark for correct use of LOAD/STOR, 1 mark for computing A^2+B, 1 mark for computing 2C*D, 1 mark for division, 1 mark for final subtraction, 1 mark for storing final answer in y.*
- *6 marks: two-address program – 1 mark for MOV to initialise, 1 mark for computing 2C, 1 mark for 2C*D, 1 mark for A^2+B, 1 mark for division, 1 mark for final subtraction into y.*
- *6 marks: three-address program – 1 mark for MUL x,C,D, 1 mark for doubling to 2C*D, 1 mark for MUL z,A,A, 1 mark for ADD z,B,z, 1 mark for DIV x,x,z, 1 mark for SUB y,A,x.*
- *1 mark: correct assembly notation (mnemonics and operand order) consistent with Table 1 for all three programs.*

Model confidence: medium.

*Model note: Figure 2 is not fully legible from the extracted text; the expression is assumed to be y = A - (2C*D)/(B + A^2). If the figure shows a different grouping (e.g., denominator B + A^2 or (B+A)^2), the programs must be adjusted accordingly. Table 1 uses inconsistent punctuation (e.g., ADD (p.q.r)); operand order is taken as dest, src1, src2 for three-address and dest, src for two-address. One-address instructions are assumed to use an implicit accumulator.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA